CSC 251 Lecture 07: Other Data Structure: Dictionaries, Sets, and Maps

Dictionaries

unique + immutable

A dictionary is an object that stores a collection of data where each element has two parts: a key and a value. Each key is mapped to a value creating a key-value pair. The **keys** can be strings, integers, floats, or tuples but NOT lists. The **values** may be objects of any type. For example:

not unique

student ID number and student name
person's name and phone number
course number and class section

Example 1a: To create a dictionary of course ID numbers and section names:

```
csc_courses = {'79543':'2023FA0 CSC-295-001','79547':'2023FA0 CSC-116-001',
'79544':'2025FA0 CSC-251-001'}
```

Example 1b: To output:

```
print(csc_courses)
```

```
{'79543':'2023FA0 CSC-295-001','79544':'2025FA0 CSC-251-001',
'79547':'2023FA0 CSC-116-001'}
```

Note: Since Python 3.6, dictionaries are stored in insertion order.

Example 1c: To retrieve an item from the dictionary:

```
print(csc_courses['79544'])
2025FA0 CSC-251-001
```

caution!

If an invalid key is entered, a `KeyError` will be returned. To avoid an error, test that the key is in the dictionary with the `in` operator:

sol'n 1

```
if '79544' in csc_courses:
    print(csc_courses['79544'])
```

Example 1d: Dictionaries are mutable objects thus you can add elements to an existing dictionary, as long as the key is unique. To add a single item, indicate dictionary and key on the left side of the assignment, values on right.

```
csc_courses['79545'] ='2024FA0 CSC-230-003'
```

OR, use the `update` method to add one or multiple key-value pairs or merge another dictionary:

```
csc_courses.update({'79545': '2024FA0 CSC-230-003'})
print(csc_courses)
```

```
{'79543':'2023FA0 CSC-295-001', '79544':'2025FA0 CSC-251-001',
'79545':'2024FA0 CSC-230-003', '79547':'2023FA0 CSC-116-001'}
```

What if you add another entry with the same key?

previous value is overwritten

Example 1e: To delete an element:

```
del csc_courses['79545']
```

Example 1f: To retrieve the number of elements in the dictionary:

```
num = len(csc_courses)
```

Example 2: To create an empty dictionary:

dict must exist to add to it

```
csc_courses = {}
```

or

```
csc_courses = dict()
```

constructor similar to list() and tuple()

Then add key-value pairs:

```
csc_courses['79545'] ='2024SP0 CSC-230-001'
csc_courses['79546'] ='2023SP0 CSC-295-004'
```

Example 3: To create a dictionary using the `fromkeys` method:

```
keys = ['a', 'b', 'c']
my_dict = dict.fromkeys(keys, 1)
print(my_dict)
```

{'a': 1, 'b': 1, 'c': 1}

What happens if no value is indicated?

Given a value of None (can't use w/ +=1)

Example 4: To create a dictionary with a **dictionary comprehension**:

```
keys = [1, 2, 3, 4]
dictComp1 = {k: k*100 for k in keys}
print(dictComp1)
{1: 100, 2: 200, 3: 300, 4: 400}
```

or

```
keys = ['A', 'B', 'C', 'D']
values = [100, 200, 300, 400]
dictComp2 = {k:v for (k,v) in zip(keys, values)}
print(dictComp2)
{'A': 100, 'B': 200, 'C': 300, 'D': 400}
```

zips into tuples of key-value pairs

each tuple used to create a pair

Example 5: A dictionary can contain values of lists or mixed types:

even other dicts

```
student_info = {'Sally Mae':[123211, '336-555-1923','BS.CSC'],
        'Fanny Mac':[109911, '919-555-8213','BS.MTH'],
        'Benny Blue':[932290, '305-123-4992','BS.ECE'] }
```

or

```
keys = ['a', 'b']
vlist = []
my_dict = dict.fromkeys(keys, vlist)
my_dict['a'].append(1)
print(my_dict)
```

So notebook → the same list for each value → not a deep copy

Other Dictionary Methods

Method	Description
clear()	Clears the contents of a dictionary.
get(key, default)	Gets the value associated with a specified key. If the key is not found, the method does not raise an exception. Instead, it returns a default value.
items()	Returns all the keys in a dictionary and their associated values as a sequence of tuples.
values()	Returns all the values in the dictionary as a sequence of tuples.
keys()	Returns all the keys in a dictionary as a sequence of tuples.
pop(key, default)	Returns the value associated with a specified key and removes that key-value pair from the dictionary. If the key is not found, the method returns a default value.
popitem()	Returns, as a tuple, the key-value pair that was last added to the dictionary. The method also removes the key-value pair from the dictionary.

Example 6: The `get` method's general format includes first a key argument then an argument default message to output if the item is not found.

dictionary`.get(`*key*`,` *default*`)`

Sol'n 2

```
csc_courses = {'79543':'2023FA0 CSC-295-001', '79544':'2025FA0 CSC-251-001',
               '79547':'2023FA0 CSC-116-001'}

course = csc_course.get('79543','Course not found')
2023FA0 CSC-295-001

course = csc_course.get('79540','Course not found')
Course not found
```

Example 7a: To iterate over a dictionary:

```
for key in student_info:
    print(key, student_info[key])

Sally Mae [123211, '336-555-1923','BS.CSC']
Fanny Mac [109911, '919-555-8213','BS.MTH']
Benny Blue [932290, '305-123-4992','BS.ECE']
```

Example 7b: To access just the keys, *dictionary*`.keys()` will return a sequence of the keys:

```
for key in csc_courses.keys():
    print(key)

79543
79544
79547
```

Example 7c: To access just the values, *dictionary.values()* will return a sequence of the values:

```
for value in csc_courses.values():
    print(value)
```

```
2023FA0 CSC-295-001
2025FA0 CSC-251-001
2023FA0 CSC-116-001
```

Example 7d: To access both the keys and values, use *dictionary.items()* to return a sequence with both:

```
for key, value in csc_courses.items():
    print(key, value)
```

```
79543 2023FA0 CSC-295-001
79544 2025FA0 CSC-251-001
79547 2023FA0 CSC-116-001
```

Example 8: To return a value associated with a specific key and remove the key-value pair from the dictionary,

dictionary.pop(key, default)

```
csc_courses = {'79543':'2023FA0 CSC-295-001', '79544':'2025FA0 CSC-251-001',
       '79547':'2023FA0 CSC-116-001'}

pop_course = F23courses.pop('79547', 'key not found')
print(pop_course)
```

```
2023FA0 CSC-116-001
```

```
print(csc_courses)
```

```
{'79543':'2023FA0 CSC-295-001', '79544':'2025FA0 CSC-251-001'}
```

Example 9:

```
# This program uses a dictionary as a deck of cards.
import random

def main():
    deck = create_deck()
    num_cards = int(input('How many cards should I deal? '))
    deal_cards(deck, num_cards)

# The create_deck function returns a dictionary representing a deck of cards.
def create_deck():
    # Create a dictionary with each card and its value stored as key-value pairs.
    deck = {'Ace of Spades':1, '2 of Spades':2, '3 of Spades':3,
            '4 of Spades':4, '5 of Spades':5, '6 of Spades':6,
            '7 of Spades':7, '8 of Spades':8, '9 of Spades':9,
            '10 of Spades':10, 'Jack of Spades':10,
            'Queen of Spades':10, 'King of Spades': 10,

            'Ace of Hearts':1, '2 of Hearts':2, '3 of Hearts':3,
            '4 of Hearts':4, '5 of Hearts':5, '6 of Hearts':6,
            '7 of Hearts':7, '8 of Hearts':8, '9 of Hearts':9,
            '10 of Hearts':10, 'Jack of Hearts':10,
            'Queen of Hearts':10, 'King of Hearts': 10,

            'Ace of Clubs':1, '2 of Clubs':2, '3 of Clubs':3,
            '4 of Clubs':4, '5 of Clubs':5, '6 of Clubs':6,
            '7 of Clubs':7, '8 of Clubs':8, '9 of Clubs':9,
            '10 of Clubs':10, 'Jack of Clubs':10,
            'Queen of Clubs':10, 'King of Clubs': 10,

            'Ace of Diamonds':1, '2 of Diamonds':2, '3 of Diamonds':3,
            '4 of Diamonds':4, '5 of Diamonds':5, '6 of Diamonds':6,
            '7 of Diamonds':7, '8 of Diamonds':8, '9 of Diamonds':9,
            '10 of Diamonds':10, 'Jack of Diamonds':10,
            'Queen of Diamonds':10, 'King of Diamonds': 10}

    return deck

# The deal_cards function deals a specified number of cards
# from the deck.
def deal_cards(deck, number):
    hand_value = 0

    if number > len(deck):
        number = len(deck)

    for count in range(number):
        card = random.choice(list(deck))
        print(card)
        hand_value += deck[card]

    # Display the value of the hand.
    print(f'Value of this hand: {hand_value}')

main()
```

unique

change value based upon game

deals a random card from desk / list

Sets

A set is a collection of unique, unordered, unchangeable, and unindexed items stored under a single variable.

- Unique implies there are no duplicate items.
- Since they are unordered, you cannot refer to them by an index or key, only by reference (set name)
- Once the set is created, you cannot change the items, but you can add and remove items.

Example 10a: To create a set, use a comma-separated sequence enclosed in curly braces:

```
languages = {"Python", "C", "Java"}
print(languages)
{'Java', 'C', 'Python'}
```

also {} like dict

Example 10b: What if you include duplicates?

```
set1 = {4, 3, 2, 1, 3, 4}
print(set1)
```

Example 10c: Since "set" is a type/object, you can use the set constructor with a tuple, list, or string:

```
set2 = set(("yes", "no", True, 1))
print(set2)
```

equivalent!

len(set2) = 3 not 4!

Example 10d: Use a set comprehension:

```
set3 = {item**2 for item in set1 if item < 20}
```

Example 11a: To iterate over the set:

```
for item in set2:
    print(item)
True
yes
no
```

Example 11b: To check if an item is present/not present use `in` and `not in`:

```
print('yes' in set2)
print('maybe' not in set2)
```

Example 11c: If you attempt to change an item:

can't be indexed!

```
set2[0] = False
TypeError: 'set' object does not support item assignment
```

Example 11d: Although you can add an item:

```
set2.add(False)
print(set2)
{False, True, 'no', 'yes'}
```

Example 11e: You can also add multiple items as long as they are passed in as an iterable:

```
set2.update(["this", "that"])
print(set2)
{False, True, 'yes', 'no', 'that', 'this'}
```

Example 11f: To remove an item:

```
set2.remove("this")
print(set2)
{False, True, 'yes', 'no', 'that'}
```

Example 11g: A better option to remove an item (in case it's not found in the set):

no errors

```
set2.discard("this")
print(set2)
{False, True, 'yes', 'no', 'that'}
```

Example 11h: To remove all items:

```
set2.clear()
print(set2)
set()
```

Set Operations with Shorthand Operators

directly calling methods

using symbols

length	`len(set1)`	
union	`set4 = set1.union(set2)`	`set4 = set1 | set2`
intersection	`set5 = set1.intersection(set2)`	`set5 = set1 & set2`
difference (in set1 but not set2)	`set6 = set1.difference(set2)`	`set6 = set1-set2`
symmetric difference (elements not shared)	`set7 = set1.symmetric_difference(set2)`	`set7 = set1 ^ set2`
subset (returns True/False)	`set1.issubset(set2)`	`set1 <= set2`
superset (returns True/False)	`set1.issuperset(set2)`	`set1 >= set2`

Advantages of Sets

- Convenience for unique elements
- Searchable
- Better performance
- Dynamic w/no overflow

Disadvantages of Sets

- Not indexable
- Memory hog
- Not appropriate for large datasets

How much memory is used for a set?

216 for a 4-int set

Maps

A Python function that applies a given function to every item in an iterable (list, tuple, etc.) and returns an iterator.

- More efficient than an explicit loop
- Applies lazy execution, thus more memory efficient
- Often used with lambda functions

General syntax:

```
map (function, iterable(s))
```

Example 12a: Used with a named function:

```
def double_plus_one(num):
    return num *2 + 1

vals = list(range(1,6))
new = map(double_plus_one, vals)

print(new)                                  <map at 0x10b7cb9d0>
print(list(new))                            [3, 5, 7, 9, 11]
```

Example 12b: Used with a built-in or imported function:

```
import math
num = [4.3, 5.99, 1.1, 10.2, -5.2]
round_up = map(math.ceil, num)
list(round_up)                              [5, 6, 2, 11, -5]
```

Example 12c: Used with a lambda function:

```
new = map(lambda x: x*2+1, vals)
```

unnamed

Example 12d: Used with multiple iterables:

```
tup = (1,2,3,4)
list1 = [10,20,30,40]
mult = map(lambda x, y: x*y, tup, list1)
list(mult)                                  [10, 40, 90, 160]
```